

Modern Cloud Computing

Lecture 8: Resource Management and Scheduling I

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- Basic Concepts
- Scaling
- Scheduling

Basic Concepts

Resource Management

Resource management affects:

- **Performance**
 - Throughput, response time, latency, deadline satisfaction
- **Cost**
 - Hardware, energy, network, migration, and management overhead

Challenges

Clouds have a very large number of shared resources.

The manager must make decisions under:

- Unpredictable user requests
- Fluctuating workloads
- Hardware and software heterogeneity
- Incomplete global state

Cloud Resource Management Goals

A cloud provider must simultaneously optimize for several goals:

User-facing goals

- Low response time
- High availability
- Predictable performance
- QoS or SLA compliance
- Elastic scaling

Provider-facing goals

- High utilization
- Low energy cost
- Low migration overhead
- Efficient placement
- Minimal SLA violations

Five Classes of Resource Management Policies

Cloud resource management policies can be loosely grouped into five broad classes:

1. **Admission control**
2. **Capacity allocation**
3. **Load balancing**
4. **Energy optimization**
5. **QoS guarantees**

These policies are coupled (improving one objective may hurt another).

Example: Consolidating workloads can save energy, but may increase contention and hurt QoS.

Admission Control

Admission control decides whether new workload should be accepted.

The goal is to avoid violating high-level system policies.

A cloud may reject or delay workload if accepting it would:

- Overload current resources
- Prevent existing tasks from finishing
- Violate already contracted SLAs

Difficulty: Admission control needs global state, but global state is hard to know accurately at cloud scale.

Capacity Allocation

Capacity allocation decides how much resource each service instance receives.

Resources include:

- CPU
- Memory
- Storage
- Network bandwidth
- Accelerator devices
- I/O capacity

Constraints:

- SLA requirements
- Resource availability
- Placement constraints
- Hardware heterogeneity
- Cost and energy goals

Load Balancing

Load balancing often means distributing work evenly.

But for energy efficiency, it may mean the opposite.

Performance view

- Spread load across servers
- Reduce hotspots
- Avoid overloaded machines
- Improve latency

Energy view

- Use fewer active servers
- Put idle servers into standby
- Reduce power cost

Tradeoff: Spreading work and consolidating work optimize different goals.

Energy Optimization

Energy optimization tries to reduce cloud operating cost and ecological impact.

Techniques include:

- Dynamic Voltage and Frequency Scaling (DVFS)
- Workload consolidation
- Turning servers to standby mode
- Energy-aware placement

DVFS: Energy vs. Performance

Lowering processor frequency and voltage reduces power consumption.

- Energy decreases substantially as clock rate decreases
- Performance decreases more slowly than energy

This creates useful operating points.

Example: At 1.8 GHz, energy may be reduced by about 18%, while performance is only about 5% lower than peak.

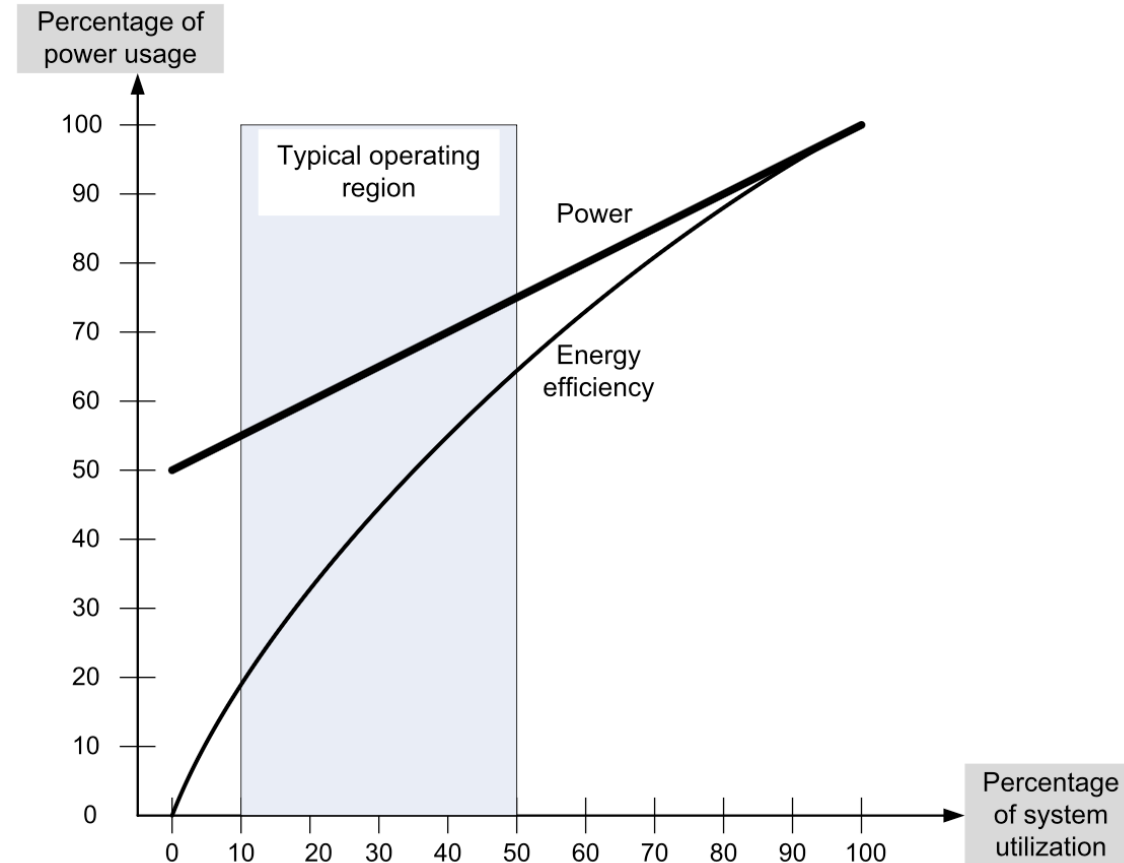
Energy-Proportional Computing

An ideal energy-proportional system:

- Uses no power when idle
- Uses little power under light load
- Gradually uses more power as load increases
- Always operates near 100% energy efficiency

Real servers are far from ideal.

Even idle systems may consume a large fraction of peak power.



Overprovisioning and Elasticity

Elasticity: additional resources are guaranteed when needed, but will be released when no longer needed.

Overprovisioning means having more capacity than normal average demand.

This enables:

- Fast response to demand spikes
- Flexible resource allocation
- The illusion of infinite resources

But it also causes:

- Low average utilization
- Poor energy efficiency
- High infrastructure cost

Typical Server Utilization Problem

The chapter emphasizes a mismatch:

- Data center servers often operate in the 10%–50% utilization range
- Idle or lightly loaded servers still consume significant power
- Energy efficiency is poor at low utilization

This motivates energy-aware resource management.

Possible strategy:

```
Monitor utilization
→ consolidate workload
→ migrate or reschedule tasks
→ place unused servers into low-power mode
→ reactivate servers when load increases
```

Resource Management Mechanisms

Mechanism	Features
Control theory	Use feedback mechanisms to guarantee stability
Machine learning	Needs training data and validation
Utility-based	Requires performance model and mechanism to connect user-level performance with cost
Market-oriented	Do not need a model of the system

Interactive vs. Non-Interactive Workloads

Resource management depends on workload type.

Interactive workloads

- Examples:
 - Web services
 - APIs
 - User-facing systems
- Need low response time
- Managed by flow control, placement, scaling

Non-interactive workloads

- Example:
 - Batch jobs
 - Analytics
 - Large computations
- Need throughput and turnaround time
- Managed mainly by scheduling

Scaling

Dynamic Application Scaling

Application demand can change over time.

- Time-of-day variation
- Monotonic growth or decline
- Predictable peaks
- Unpredictable spikes
- Failures that force relocation

Cloud systems need scaling mechanisms that respond to these changes without destabilizing the system.

Vertical vs. Horizontal Scaling

Scaling strategy	Meaning	Example	Tradeoff
Vertical scaling	Add more resources to existing VMs	More CPU share or move to stronger server	May require migration; limited by one machine
Horizontal scaling	Add more VMs or instances	Add more backend servers	Needs load balancing; may increase communication

Horizontal scaling is more common because many cloud applications can be replicated across instances.

Designing Applications for Scaling

Applications should be designed with scaling in mind.

A scalable application should:

- Support adding/removing workers
- Partition work dynamically
- Load balancing across instances

The resource manager cannot magically scale an application that was designed as a single fixed unit.

Auto-Scaling Requirements

Auto-scaling needs two things:

1. A resource pool

- Resources can be allocated and released on demand

2. A monitoring and control loop

- *Sensors* observe system state
- *Controllers* decide scaling actions
- *Actuators* create, remove, or migrate resources

Common metrics:

- CPU utilization
- Request rate
- Queue length
- Response time

Auto-Scaling Control Loop

Sensors

→ measure CPU, memory, queue length, response time

Monitor

→ compares current state against thresholds or models

Controller

→ decides scale up, scale down, migrate, or hold

Actuator

→ starts/stops VMs, changes CPU share, moves workload

Danger: Control actions can oscillate if thresholds are poor or reaction delay is ignored.

Control Theory for Resource Management

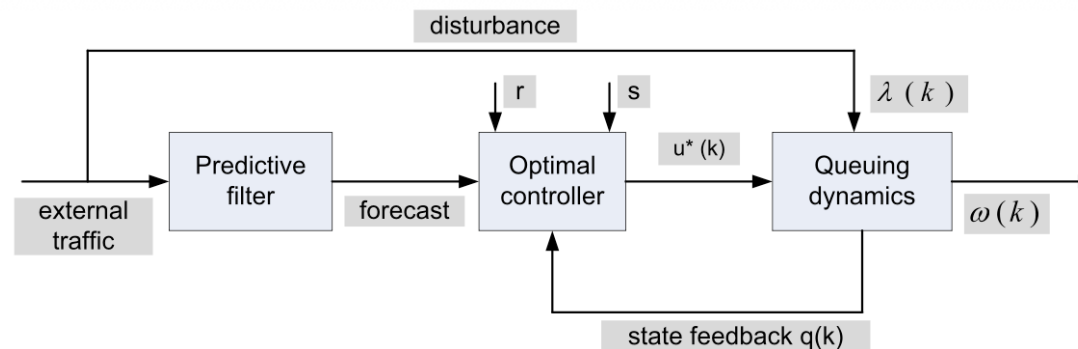
Control theory uses feedback to regulate system behavior.

For resource management, the controller can adjust:

- CPU frequency
- Number of active servers
- Number of VMs
- Admission rate
- Load distribution

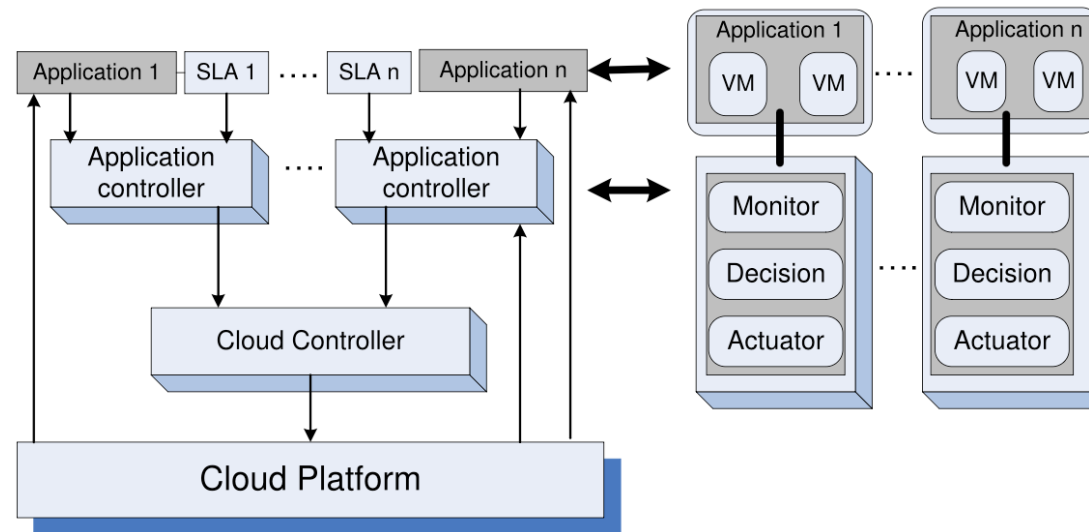
The controlled output may be:

- Queue length
- Response time
- Power consumption
- SLA violation rate



Two-Level Resource Allocation Architecture

- **Application controllers**
 - Understand application-level needs and SLA targets
- **Cloud controllers**
 - Understand platform-level resource availability and placement



Unstable Controllers

A resource-control system can become unstable due to:

1. Delay

- The effect of a control action appears later

2. Granularity

- Adding or removing one VM may be too large a change

3. Oscillation

- The controller repeatedly scales up and down

Example: If high and low thresholds are too close, a workload can bounce between scale-up and scale-down actions.

Threshold-Based Control

A threshold triggers a change in system behavior.

Examples:

- CPU utilization $> 80\%$ $\rightarrow$ add a VM
- Queue length $>$ threshold $\rightarrow$ increase capacity
- Utilization $< 20\%$ $\rightarrow$ remove a VM

Thresholds can be:

- **Static:** fixed once
- **Dynamic:** based on recent measurements or multiple parameters

Proportional Thresholding

Proportional thresholding uses dynamic high and low thresholds.

A simplified algorithm:

```
Compute high and low thresholds from recent CPU-utilization history
  High threshold = average of the maximum processor utilization over the process history.
  Low threshold = average of the minimum processor utilization over the process history.

If current average CPU utilization > high threshold:
  request additional VMs

If current average CPU utilization < low threshold:
  release one VM
```

Scheduling

Scheduling in Clouds

Scheduling is a core part of resource management.

It appears at multiple levels:

- Cluster shared among users and jobs
- Server shared among VMs
- VM shared among applications
- Application split into threads or tasks
- Network link multiplexed among flows

Schedulers should be:

- Efficient
- Fair
- Starvation-free
- Suitable for the workload type

Scheduling Objectives

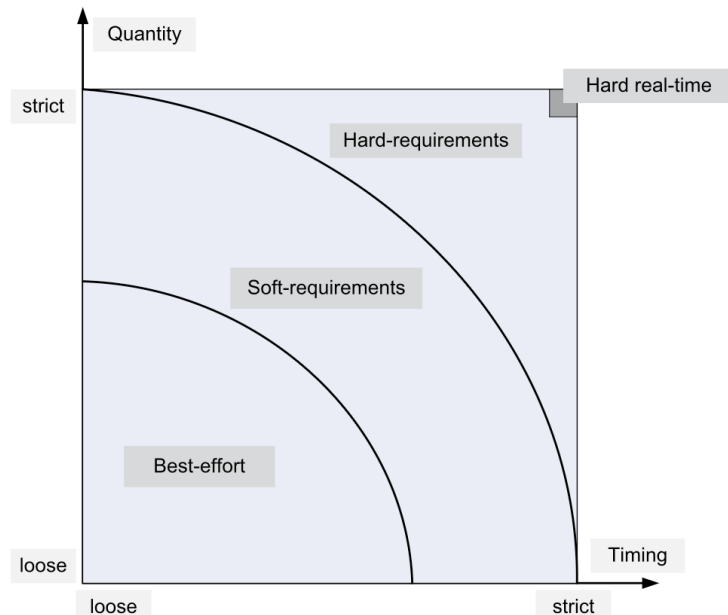
System type	Scheduler objective
Batch system	Maximize throughput and minimize turnaround time
Interactive system	Minimize response time and tail latency
Real-time system	Meet deadlines and be predictable
Mixed workload cloud	Balance throughput, latency, fairness, and QoS

Problem: Different objectives can conflict.

Resource Requirement Policies

Scheduling policies can be understood along two dimensions:

- **Amount of resources allocated**
- **Timing when access to resources is granted**



Broad policy classes:

- **Best effort**
 - No strict resource or timing guarantees
- **Soft requirements**
 - Statistical guarantees
- **Hard requirements**
 - Strict timing and precise resource guarantees

Fairness in Scheduling

Distribute resources fairly among competing users or workloads

- Prevent a single workload from monopolizing resources; avoid starvation of other workloads
- Fairness does not necessarily mean equal allocation
 - Resources may be allocated according to priorities or weights
- Fairness may consider multiple resource types
 - e.g., CPU and memory
- Fairness is often evaluated over time
 - Temporary imbalance is acceptable if long-term shares are fair
- Common approaches:
 - Fair sharing
 - Weighted fair sharing
 - Max-min fairness
 - Dominant Resource Fairness (DRF)

Fairness: Max-Min Fairness

Max-min fairness tries to improve the allocation of the least-served user first.

Assume total amount of resources is B . Each user requests resource b_i and gets B_i .

A fair allocation should satisfy:

- $C_1: B_i \leq b_i$
 - A user does not receive more than requested
- C_2 : If minimum allocation of any user is B_{min} , no other allocation satisfying C_1 has a higher B_{min} than the current allocation
 - The minimum allocation is as large as possible
- C_3 : After removing the minimum-allocated user ($B := B - B_{min}$), the same logic applies recursively

Intuition: Protect small users from being starved by large users.

Dominant Resource Fairness (DRF)

DRF extends fair sharing to workloads that require **multiple resource types**.

- Different workloads may have different resource bottlenecks
 - e.g., CPU-intensive vs. memory-intensive workloads
- **Dominant resource**
 - The resource for which a workload requires the **largest fraction of total cluster capacity**
- **Dominant share**
 - The fraction of the dominant resource allocated to a user/workload
- DRF allocates resources to **equalize dominant shares**
 - Repeatedly allocate resources to the user with the smallest dominant share

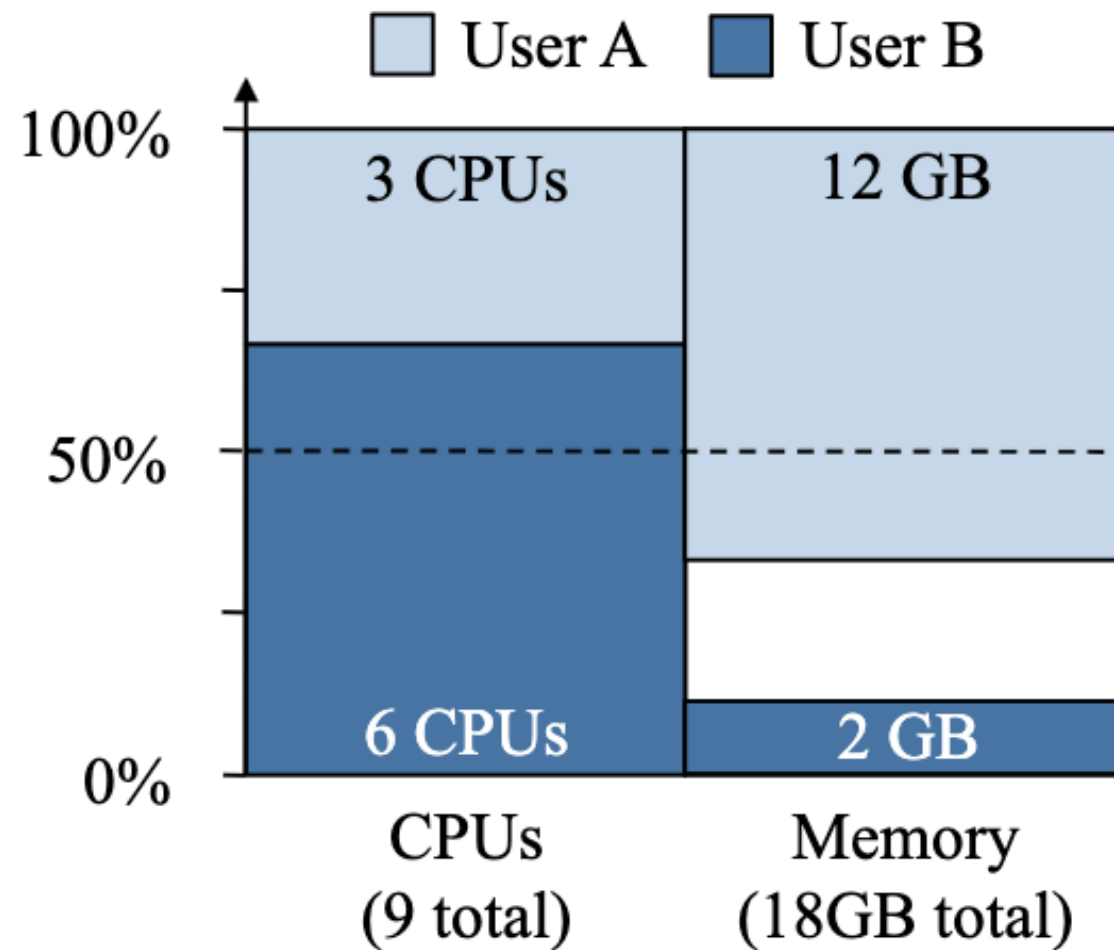
DRF Example

Cluster capacity: **9 CPUs + 18 GB memory**

- Each task from user A requires **1 CPU + 4 GB**
 - CPU share = $1/9$, Memory share = $2/9$ → **Dominant share = $2/9$**
- User B receives **3 CPUs + 1 GB**
 - CPU share = $1/3$ Memory share = $1/18$ → **Dominant share = $1/3$**
- DRF: 3 tasks for User A, 2 tasks for User B

Key idea: Fairness is based on each workload's most heavily demanded resource.

Ghods et al., "Dominant Resource Fairness: Fair Allocation of Multiple Resource Types," NSDI 2011.



Common Scheduling Algorithms

Algorithm	Idea	Strength	Weakness
FCFS (<i>First Come First Serve</i>)	Run in arrival order	Simple	Can suffer from long jobs blocking short ones
Round robin	Time-slice each task	Responsive	Context-switch overhead
SJF (<i>Shortest Job First</i>)	Run shortest job first	Good average turnaround	Requires job-length estimate
Priority	Run highest priority	Supports differentiated service	Starvation risk

Delay Scheduling

Delay scheduling balances fairness and data locality.

Main idea:

- A job may wait briefly for a better local slot
- Waiting improves chance of node-local or rack-local execution
- If it waits too long, the scheduler relaxes locality preference

This is useful for data-intensive systems like Hadoop.

Tradeoff: Relax strict fairness to improve locality and throughput.

Zaharia, Matei, et al. "Delay scheduling: a simple technique for achieving locality and fairness in cluster scheduling." Proceedings of the 5th European conference on Computer systems. 2010.

Hierarchical Scheduling

Cloud scheduling is often hierarchical.

Example hierarchy:

```
Cluster
→ organization queue
→ user pool
→ job
→ task
```

Benefits:

- Reflects organizational policies
- Supports quotas and priorities
- Allows local policy within each group
- Scales better than one flat global policy

Hadoop Fair Scheduler (HFS)

HFS provides **fair resource sharing among users** through a hierarchical scheduling policy.

- **Two-level scheduling**

- Level 1: allocate task slots to **pools of jobs** using fair sharing
- Level 2: allocate each pool's slots to **jobs within the pool**

- **User-controlled scheduling**

- Each pool can choose its internal scheduling policy
- Supports **FIFO** or **fair sharing**

- **Predictable turnaround time**

- Each pool has a guaranteed minimum share of slots
- Uses **minimum share timeout** and **fair share timeout**
- Long-running or buggy tasks may be killed when timeouts occur

- **Hierarchical fair sharing**

- Pools below their minimum share receive priority
- Other pools are ordered using weighted fair sharing

HFS: Delay Scheduling for Data Locality

HFS may **delay task assignment** to wait for a more local execution opportunity.

- Each job maintains a **locality level** and **wait time**
- **Level 0 — Node-local**
 - Initially, only run a task if its data is on the available node
- **Level 1 — Rack-local**
 - After waiting at least W_1 , allow tasks whose data is in the same rack
- **Level 2 — Off-rack**
 - After waiting an additional W_2 , allow any unlaunched task
- When a more-local task is successfully scheduled:
 - Reset the job's wait time
 - Move the job back to a better locality level

Wait briefly for locality, but gradually relax locality requirements to avoid excessive delay.

Data-Aware Scheduling

Data locality matters for I/O-intensive applications.

A data-aware scheduler tries to place computation near data.

Benefits:

- Less network traffic
- Lower data transfer time
- Better throughput
- Less interference with other jobs

Apache Capacity Scheduler (MapReduce Scheduler for Hadoop)

The **Capacity Scheduler** supports multiple queues with guaranteed shares of cluster resources.

- Each queue is guaranteed a fraction of the cluster capacity
- Unused capacity can be temporarily allocated to other queues
 - Excess resources can later be reclaimed to satisfy guarantees
- Jobs within a queue can have different priorities
 - Higher-priority jobs receive resources first
- No preemption based on job priority
 - A running job will not be preempted by a higher-priority job
- Per-user resource limits can be enforced within each queue
- Jobs can request more memory than the default
 - Tasks are assigned only to TaskTrackers with sufficient memory

Capacity Scheduler: Scheduling

When a TaskTracker becomes available:

1. Select a queue

- Prioritize queues that need to reclaim resources
- Otherwise, select the queue with the lowest ratio of:
running slots / guaranteed capacity

2. Select a job

- Based on submission time and priority

3. Select a task

- Choose a task from the selected job to execute

Capacity Scheduler: Resource Reclamation

- A queue can reclaim resources when:
 - It has pending tasks, and
 - Other queues are using part of its guaranteed capacity
- Resources should be restored within the configured **reclaim time**
- If resources are not returned in time:
 - The scheduler kills the tasks that started most recently

Comparing Scheduling Ideas

Scheduler idea	Main target	Key tradeoff
FCFS / RR / priority	Basic best-effort scheduling	Simple but limited QoS control
Delay scheduling	Data locality + fairness	Locality improves by relaxing fairness briefly
Capacity scheduler	Organizational sharing	Reserved capacity vs. utilization

Scheduling Challenges

Cloud schedulers face several tensions:

- Locality vs. fairness
- Energy vs. performance
- Utilization vs. isolation
- Low latency vs. high throughput
- Centralized control vs. scalability
- Exact optimization vs. fast decisions

Takeaways

- Cloud resource management is a multi-objective optimization problem.
- Accurate global state is difficult to maintain at cloud scale.
- Energy efficiency motivates consolidation, but consolidation can hurt QoS.
- Auto-scaling requires monitoring, control, and stable thresholds.
- Control theory, ML, utility models, and market mechanisms each have strengths and limits.
- Scheduling must balance fairness, locality, throughput, latency, and policy constraints.